

Model Comparison on the Cylinder Graph HMM

Claude & C. L.

February 11, 2026

1 Introduction

A Hidden Markov Model (HMM) is a stochastic process in which an unobservable (hidden) Markov chain $\{S_t\}$ over a finite state space generates an observed sequence $\{X_t\}$ through state-dependent emission distributions. At each time step the chain transitions from state s to state s' with probability $T(s, s')$ and emits an observable token v with probability $O(v | s')$. Because the hidden states are not directly accessible, a predictor must infer the latent state from the history of observations alone.

In this report we study how small transformer language models learn to perform this inference on a structured HMM whose hidden-state graph has the topology of a discrete cylinder (Section 2). We first establish information-theoretic baselines—the entropy rate, the Bayes-optimal predictor at finite context, and n -gram models—against which transformer performance can be measured (Section 3). We then describe the transformer architectures and training procedure (Section 4), present results on scaling, architecture, and embedding structure (Section 5), and conclude with open questions (Section 6).

2 The Cylinder Graph HMM

We consider an HMM whose hidden-state graph has the topology of a discrete cylinder with *width* $n = 6$ (nodes per level) and *depth* $D = 3$ (number of levels), giving $n \times D = 18$ hidden states in total.

2.1 Transition structure

Each level $l \in \{0, 1, 2\}$ contains $n = 6$ nodes arranged in a ring. From a node (l, j) at depth l and angular position j the chain can jump to three neighbours:

- **Nearest neighbour (NN):** $(l, j+1 \bmod n)$ with weight w , where $w \sim \text{Uniform}(0, 1-p)$;
- **Next-nearest neighbour (NNN):** $(l, j+2 \bmod n)$ with weight $1 - p - w$;
- **Next layer:** $((l+1) \bmod D, j)$ with fixed probability $p = 0.1$.

The intra-layer weights w are drawn independently for each node from $\text{Uniform}(0, 1-p)$ once at initialisation (seed 42) and held fixed thereafter. The inter-layer probability p controls how often the chain cycles through the depth levels.

2.2 Emission structure

The vocabulary has $|V| = D \times K = 3 \times 16 = 48$ tokens, partitioned into three clusters of 16 tokens each (one cluster per depth level). Each hidden state at depth l emits tokens only from cluster l , with emission probabilities drawn from a symmetric Dirichlet distribution with concentration parameter $\alpha = 0.3$. The small α makes the per-state distributions sparse, so different states at the same depth emit noticeably different token profiles.

3 Information-Theoretic Baselines

3.1 Entropy rate

The *entropy rate* of a stationary stochastic process $\{X_t\}$ is

$$h = \lim_{T \rightarrow \infty} \frac{1}{T} H(X_1, \dots, X_T) = \lim_{T \rightarrow \infty} H(X_T \mid X_1, \dots, X_{T-1}). \quad (1)$$

For the second equality, we have used the chain rule of entropy. The entropy rate of a stochastic process is the irreducible uncertainty per token for a predictor with access to the infinite past, and therefore serves as the information-theoretic lower bound on the cross-entropy loss achievable by any model.

For an HMM with transition matrix T and emission matrix O , the entropy rate can be estimated empirically by generating a long sequence $(x_1, \dots, x_N)$, running the forward algorithm to compute the predictive probabilities $P(x_t \mid x_1, \dots, x_{t-1})$, and averaging:

$$\hat{h} = -\frac{1}{N - N_{\text{burn}}} \sum_{t=N_{\text{burn}}+1}^N \log P(x_t \mid x_1, \dots, x_{t-1}), \quad (2)$$

where the first N_{burn} tokens are discarded to allow the belief state to converge from an arbitrary prior. For the cylinder graph HMM with the parameters specified above, this yields $\hat{h} \approx 2.44$ nats/token.

3.2 Bayes-optimal loss at finite context

The entropy rate is the minimum achievable loss for a predictor with access to the *infinite* past. A transformer with context window T , however, can only condition on the most recent T tokens. To disentangle the loss due to *finite context* from the loss due to *limited model capacity*, we compute the Bayes-optimal next-token loss at finite context: the best any predictor can do when restricted to a window of length k .

Let $\{X_t\}$ be the observed token process generated by the HMM with hidden states $\{S_t\}$, transition–emission matrices $E[v, s, s'] = P(X_t=v, S_t=s' \mid S_{t-1}=s)$, and stationary distribution π .

Given a context window $(x_1, \dots, x_k)$, the Bayes-optimal prediction for the next token is

$$P(X_{k+1}=v \mid x_1, \dots, x_k) = \sum_{s, s'} \alpha_k(s) E[v, s, s'], \quad (3)$$

where α_k is the posterior (belief state) over hidden states after processing the context. The belief state is computed recursively via the forward algorithm. Initialise from the stationary prior $\alpha_0(s) =$

$\pi(s)$. Upon observing token x_t , update:

$$\tilde{\alpha}_t(s') = \sum_s \alpha_{t-1}(s) E[x_t, s, s'], \quad \alpha_t(s') = \frac{\tilde{\alpha}_t(s')}{\sum_{s''} \tilde{\alpha}_t(s'')}. \quad (4)$$

The normalising constant $Z_t = \sum_{s'} \tilde{\alpha}_t(s')$ equals the predictive probability $P(x_t | x_1, \dots, x_{t-1})$, so the conditional log-likelihood of the context decomposes as $\log P(x_1, \dots, x_k) = \sum_{t=1}^k \log Z_t$.

After k update steps the belief α_k encodes all information that the context provides about the current hidden state. Plugging it into Eq. (3) yields the Bayes-optimal predictive distribution and hence the minimum achievable loss at context length k :

$$L^*(k) = \mathbb{E} \left[-\log P(X_{k+1} | X_1, \dots, X_k) \right], \quad (5)$$

where the expectation is over sequences drawn from the HMM.

For a *fixed* context window, each length- k chunk is treated independently: the belief is reset to π at the start of every window. This is exactly the information available to a transformer with context length k . For the *infinite-context* baseline, the forward algorithm runs over the entire sequence without resetting, so the belief accumulates information from arbitrarily far in the past. The difference

$$\Delta(k) = L_{\text{fixed}}^*(k) - L_{\infty}^* \quad (6)$$

isolates the cost of the finite context window. As k grows beyond the mixing time of the chain, $\Delta(k) \rightarrow 0$.

The cross-entropy loss of a trained transformer $\hat{L}$ can therefore be decomposed as

$$\underbrace{\hat{L}}_{\text{transformer loss}} = \underbrace{h}_{\text{entropy rate}} + \underbrace{\Delta(k)}_{\text{finite-context cost}} + \underbrace{D_{\text{KL}}}_{\text{model capacity gap}}, \quad (7)$$

where $D_{\text{KL}} = \hat{L} - L_{\text{fixed}}^*(k) \geq 0$ measures the sub-optimality of the transformer relative to the Bayes-optimal predictor at the same context length.

3.3 N -gram baseline

An n -gram model estimates the next-token distribution by conditioning on a fixed-length context of $n-1$ preceding tokens. The maximum likelihood estimate is

$$\hat{P}_{\text{MLE}}(w | w_{t-n+1}, \dots, w_{t-1}) = \frac{C(w_{t-n+1}, \dots, w_{t-1}, w)}{C(w_{t-n+1}, \dots, w_{t-1})}, \quad (8)$$

where $C(\cdot)$ denotes the count of the n -gram (or $(n-1)$ -gram context) in the training corpus. When a context has never been observed, we back off to shorter contexts until a match is found (see Appendix A for smoothing and backoff details). The implementation is in `src/ngram_baseline.py`.

3.4 Summary of baselines

Table 1 summarises the theoretical and empirical baselines.

Baseline	Loss (nats)	Description
Entropy rate h	≈ 2.44	Infinite-context lower bound
Bayes-optimal $L^*(k=16)$	≈ 2.47	Best fixed-context ($k=16$) predictor
Best n -gram (trigram)	2.57	Best count-based model (test CE)
Bigram	2.67	Single-token context baseline
Unigram	3.65	No context (marginal frequencies)

Table 1: Information-theoretic and count-based baselines for the cylinder graph HMM. The finite-context cost $\Delta(16) \approx 0.03$ nats is small, indicating that the context window $k = 16$ is already sufficient for near-optimal inference.

4 Transformer Architectures

We train causal transformer language models (implemented via TransformerLens `HookedTransformer`) to predict the next token in sequences of length $T = 16$ generated by the cylinder HMM.

The hyperparameter sweep covers:

Hyperparameter	Values
Number of layers L	2, 4
Embedding dimension d	4, 8, 16, 32, 48, 64
Number of heads H	1, 2, 4
Architecture	attention-only, full (attention + MLP)
Normalisation	none, LayerNorm

All models use ReLU activations, tied input/output embeddings, and $d_{\text{mlp}} = 4d$ for full architectures. Training uses AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01, learning rate 10^{-3}) with batch size 128 for 10 epochs. A total of 135 models were trained on a dataset of ~ 618 k sequences (10^7 tokens).

5 Results

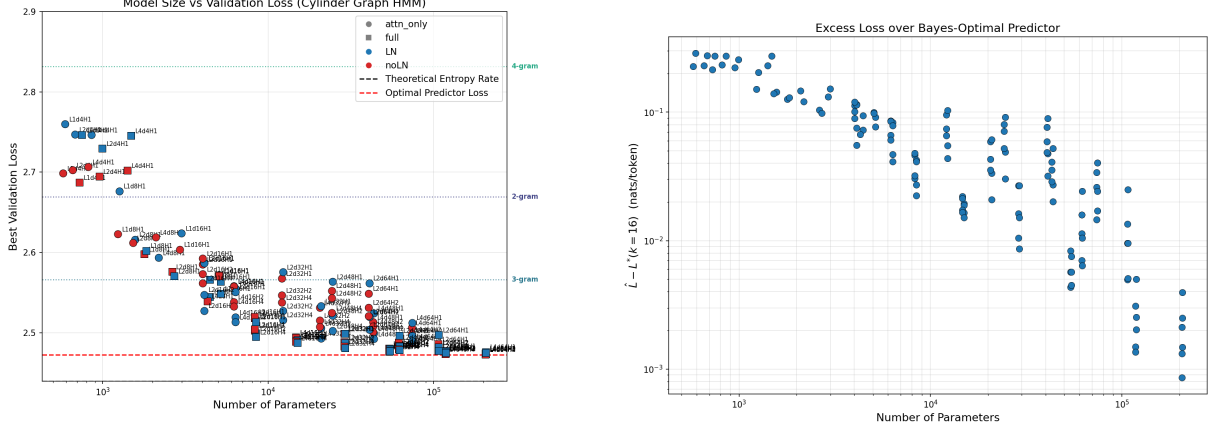
5.1 Model size vs. validation loss

Figure 1 plots the best validation loss achieved by each model against its parameter count.

Validation loss decreases roughly log-linearly with parameter count up to $\sim 10^5$ parameters, after which gains diminish. At every scale, full transformers (with MLP layers) outperform their attention-only counterparts, and all top-10 models use $L = 4$ layers. LayerNorm has a negligible effect compared to architecture and scale. The best model (L4, $d=64$, $H=4$, full, no-LN; ~ 206 k parameters) reaches ≈ 2.473 nats/token. By the decomposition in Eq. (7), most of the 0.034 nat gap above the entropy rate is attributable to the finite context window ($\Delta(16) \approx 0.03$ nats), leaving only ~ 0.004 nats of model capacity gap.

5.2 N -gram comparison

Table 2 reports the training and test cross-entropy loss for n -gram orders 1 through 6 (without regularisation; see Appendix A for smoothed results).



(a) Validation loss vs. parameter count. Circles: attention-only; squares: full (attention + MLP). Blue: LayerNorm; red: no normalisation. Dashed lines mark the entropy rate $h \approx 2.44$ nats/token, the Bayes-optimal loss $L^*(k=16) \approx 2.47$ nats, and the best n -gram baseline (trigram, 2.57 nats).

(b) Excess loss $\hat{L} - L^*(k=16)$ on a log-log scale. All architectural distinctions are suppressed to highlight the overall scaling trend with model size.

Figure 1: Model size versus validation loss for 135 transformer configurations on the Cylinder Graph HMM. (a) Absolute validation loss. (b) Excess loss over the Bayes-optimal predictor at context length $k = 16$.

n	Train CE	Test CE	# Contexts
1	3.654	3.652	1
2	2.667	2.669	48
3	2.551	2.566	1 540
4	2.470	2.832	39 531
5	2.236	4.781	443 415
6	1.713	8.618	1 953 864

Table 2: N -gram cross-entropy (nats/token) without regularisation. For $n \geq 4$, test loss diverges due to overfitting. The best test performance is the trigram ($n = 3$).

The largest drop in cross-entropy occurs from the unigram to the bigram ($3.65 \rightarrow 2.67$ nats), reflecting the strong first-order correlations induced by the HMM’s clustered emission structure: observing a single token already reveals the current depth level, narrowing the effective vocabulary from 48 to 16. The trigram provides a modest further gain, approaching the Bayes-optimal loss. The best n -gram model (trigram, test CE = 2.57) is outperformed by transformers with embedding dimension $d \geq 16$, suggesting that these models learn to approximate belief-state inference beyond simple token co-occurrence statistics. Conversely, very small transformers ($d = 4$) fail to beat the trigram, indicating that a minimum model capacity is needed before the transformer can surpass what a count table captures directly.

5.3 Embedding structure

Figure 2 shows the cosine-similarity matrix and ℓ_2 norms of the learned token-embedding vectors from a representative model. The 3×3 block structure in the similarity matrix directly mirrors the depth-level clustering of the vocabulary: tokens within the same depth cluster are mapped to similar embedding directions, while cross-cluster pairs are near-orthogonal or anti-correlated. This confirms that the model discovers the latent depth structure without supervision.

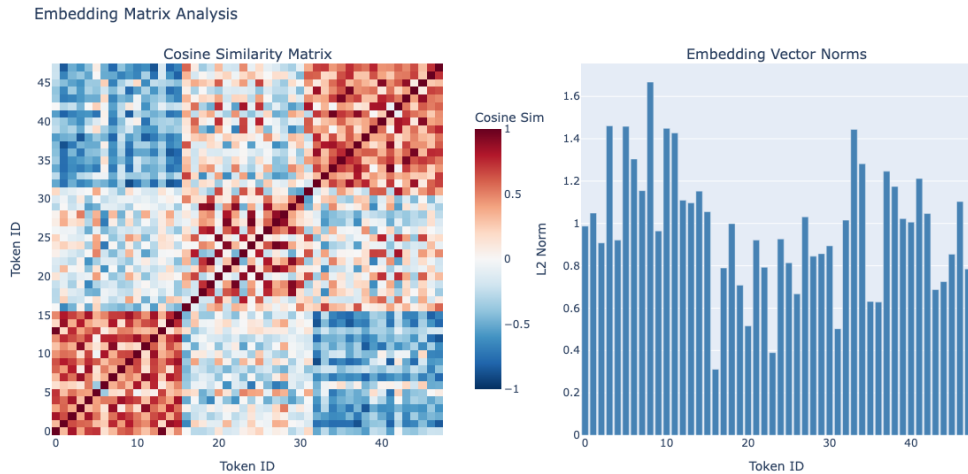


Figure 2: Left: cosine similarity matrix of the $48 \times d$ embedding matrix. The visible 3×3 block structure corresponds to the three depth levels. Right: ℓ_2 norm of each token’s embedding vector.

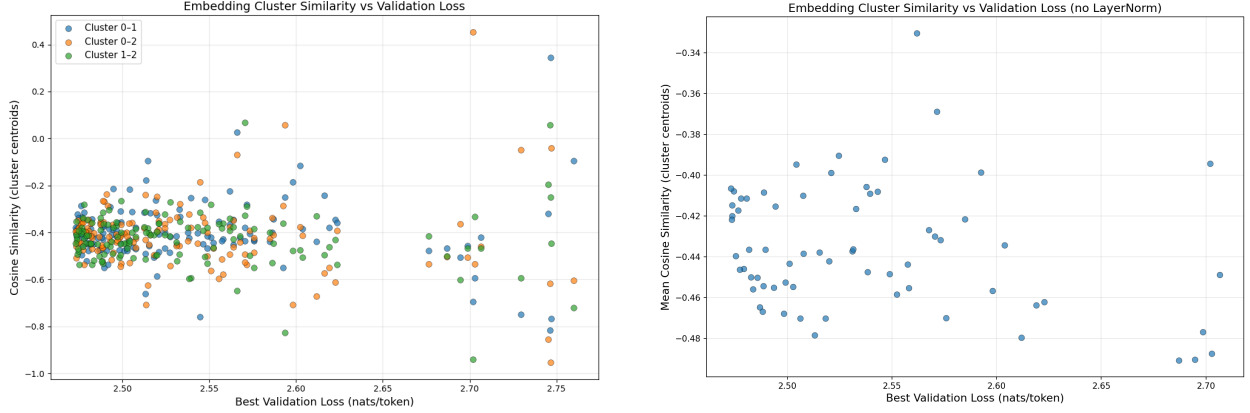
Cluster similarity across architectures To quantify how consistently the embedding geometry reflects the depth-level structure, we compute the cosine similarity between cluster centroids across all 135 trained models. For each model we extract the embedding matrix $W_E \in \mathbb{R}^{48 \times d}$ and compute three centroids

$$\mathbf{c}_l = \frac{1}{16} \sum_{v=16l}^{16l+15} W_E[v, :], \quad l \in \{0, 1, 2\}, \quad (9)$$

followed by the three pairwise cosine similarities $\text{sim}(\mathbf{c}_0, \mathbf{c}_1)$, $\text{sim}(\mathbf{c}_0, \mathbf{c}_2)$, and $\text{sim}(\mathbf{c}_1, \mathbf{c}_2)$.

Figure 3 shows these similarities as a function of validation loss. All three pairs are predominantly negative (mean ≈ -0.4), confirming that the learned embeddings separate the three depth clusters into roughly opposing directions in embedding space. Models that achieve low validation

loss (left side of the plot) cluster tightly in the range -0.35 to -0.50 , while poorly performing models exhibit much higher variance—some cluster pairs even become positively correlated. This suggests that learning to separate the depth clusters in embedding space is necessary for good predictive performance, but the degree of separation saturates once the model is sufficiently expressive.



(a) All three pairwise cosine similarities between cluster centroids, plotted for each of the 135 trained models. Poorly performing models show high variance, with some cluster pairs becoming positively correlated.

(b) Mean of the three pairwise similarities per model, restricted to architectures without Layer-Norm (67 models). The trend is flat, with most models in $[-0.49, -0.35]$.

Figure 3: Cosine similarity between depth-cluster centroids of the embedding matrix versus best validation loss. (a) Per-pair similarities for all models. (b) Mean similarity for noLN models only.

6 Summary and Open Questions

The cylinder graph HMM provides a controlled testbed for studying how transformers learn to perform approximate Bayesian filtering over a structured latent space. Larger, deeper, full-architecture models approach but do not reach the entropy rate, suggesting fundamental limits tied to the autoregressive architecture and finite context.

Open directions include:

- Characterising the belief-state geometry learned by the residual stream (cf. the fractal structure observed in the belief-state notebooks).
- Ablation studies on context length to quantify the loss attributable to finite T .
- Mechanistic interpretability of attention heads—do specific heads specialise in tracking depth transitions versus intra-layer dynamics?

A N -gram Smoothing and Backoff Details

Laplace smoothing The MLE assigns zero probability to any n -gram unseen in training, producing infinite cross-entropy on test data. Laplace (add- k) smoothing regularises the estimate by adding k pseudocounts to every possible continuation:

$$\hat{P}_k(w \mid \mathbf{c}) = \frac{C(\mathbf{c}, w) + k}{C(\mathbf{c}) + k|V|}, \quad (10)$$

where $\mathbf{c}$ is the $(n-1)$ -token context and $|V| = 48$ is the vocabulary size. Setting $k = 0$ recovers the MLE, while $k \rightarrow \infty$ yields a uniform distribution over the vocabulary. Setting $k = 1$ (standard Laplace smoothing) has a Bayesian interpretation: it corresponds to placing a symmetric $\text{Dir}(1, \dots, 1)$ prior (uniform over the probability simplex) on the next-token distribution for each context. The posterior mean after observing counts $C_1, \dots, C_{|V|}$ is

$$\mathbb{E}[p_w \mid \text{data}] = \frac{C_w + 1}{\sum_{j=1}^{|V|} C_j + |V|}, \quad (11)$$

which is exactly Eq. (10) with $k = 1$.

Backoff When an $(n-1)$ -gram context has never been observed in training, the smoothed estimate degenerates to $1/|V|$. To avoid this, we employ a simple backoff strategy: if the full $(n-1)$ -gram context is absent from the count table, we fall back to the $(n-2)$ -gram model, then the $(n-3)$ -gram, and so on down to the unigram. The first context length that appears in the training counts is used with its smoothed estimate.

Data sparsity at high order The number of possible $(n-1)$ -gram contexts grows as $|V|^{n-1}$. For $|V| = 48$, a 6-gram model has $48^5 \approx 2.5 \times 10^8$ possible contexts, far exceeding the $\sim 10^7$ training tokens. Most high-order contexts are therefore observed very few times, and the Laplace pseudocounts ($k \cdot |V| = 48$ per context) dominate the actual counts, pushing the smoothed distribution toward uniform and *increasing* the cross-entropy. This explains the well-known bias-variance trade-off in n -gram models: higher orders reduce bias (they can capture longer dependencies) but increase variance (most contexts are poorly estimated).

Effect of smoothing on high-order n -grams Table 3 compares two smoothing settings.

n	$k = 1$ (Laplace)		$k = 10^{-10}$ (no reg.)		# Contexts
	Train CE	Test CE	Train CE	Test CE	
1	3.654	3.652	3.654	3.652	1
2	2.667	2.669	2.667	2.669	48
3	2.554	2.560	2.551	2.566	1 540
4	2.534	2.574	2.470	2.832	39 531
5	2.644	2.793	2.236	4.781	443 415
6	2.849	3.127	1.713	8.618	1 953 864
HMM entropy rate			2.5342 nats		

Table 3: N -gram cross-entropy (nats/token) under two smoothing regimes.

The two regimes agree closely for $n \leq 3$ where data is plentiful, but fail in qualitatively different ways at high orders.

With strong smoothing ($k = 1$), the Laplace pseudocounts add $k \cdot |V| = 48$ to the denominator of every context. For a 5-gram context observed an average of ~ 23 times, the 48 pseudocounts exceed the real data, pushing the smoothed distribution toward uniform and inflating both training and test cross-entropy.

With near-zero smoothing ($k = 10^{-10}$), the estimates are effectively pure maximum likelihood. Training cross-entropy continues to decrease monotonically—from 2.47 nats at $n = 4$ down to

1.71 nats at $n = 6$ —because more specific contexts yield sharper predictions on the data they were estimated from. However, test cross-entropy diverges catastrophically: 2.83 nats at $n = 4$, 4.78 at $n = 5$, and 8.62 at $n = 6$. The mechanism is that when a high-order context *was* observed in training but a particular next token never followed it, the smoothed probability is $\hat{P} \approx k/C(\mathbf{c}) \approx 10^{-10}$, contributing $-\log(10^{-10}) \approx 23$ nats for that single prediction. Even a small fraction of such events dominates the average.

In summary, the two smoothing regimes bracket the bias–variance trade-off: $k = 1$ over-regularises (high bias), while $k = 10^{-10}$ under-regularises (high variance). More sophisticated smoothing methods (e.g. Kneser–Ney) handle this trade-off more gracefully by redistributing probability mass based on the diversity of contexts in which each token appears.